

实验 1. Shell 基础命令

一、实验目的:

使学生通过实验进一步巩固并熟练掌握 Shell 基础命令的使用方法。

二、实验要求:

1. 掌握 Linux 系统的目录结构。
2. 掌握文件路径的表达方法。
2. 掌握文件与目录的基本操作。
4. 学会阅读命令的联机帮助文档。

三、实验内容

1. 准备实验环境

启动 Linux 系统，若系统未安装窗口系统，则直接登录系统；若安装了图形用户界面，登录系统后，首先打开终端，例如，赋予 Ubuntu 发行版，可使用快捷键 `ctrl+alt+t`。

2. 显示当前登录用户和工作目录

```
$ whoami           // 显示登录用户
$ pwd              // 显用户当前工作目录
```

3. 观察根目录和个子目录

```
$ ls /              // 观察根目录结构
$ ls -lR /usr        // 遍历目录，可按 ctrl+c 结束进程
$ ls -l /usr/        // 观察标准函数库安装路径
```

4. 观察命令的种类和文件的类型

```
# type cd           // 内部命令
$ type ls           // 外部命令
$ file /bin/pwd      // 可执行文件
$ file /usr/include/include/stdio.h // 文本文件
```

5. 查看文本文件

```
$ cat /usr/include/stdio.h // 显示文本文件的内容
$ more /usr/include/stdio.h // 分页显示
$ head -20 /usr/include/stdio.h // 显示文件的前 20 行
```

6. 目录和文件操作

```
$ cd ~              // 进入用户主目录
# mkdir testdir     // 创建新目录
$ cd testdir        // 进入 testdir 目录
$ cat >test         // 编辑完成后按 ctrl+d 结束编辑
$ cat test          // 浏览文件内容
$ cp test demo      // 为 test 文件创建副本 demo
$ cd ..             // 退回至父目录
$ cp -rf testdir backdir // 为 testdir 目录建立副本 backdir
$ rm -rf testdir    // 删除原目录 testdir
$ mv backdir testdir // 恢复原目录 testdir
```

7. 硬链接和符号链接

```
$ cd testdir        // 重新进入 testdir 目录
$ ln demo hdemo     // /为 demo 创建硬链接
```

`$ ln -s test stest` //为文件 `test` 建立符号链接

`$ ls -la` // 观察文件的引用次数

8. 使用联机帮助

`$ man ls` // 阅读 `ls` 命令的联机帮助文档

四、思考题

1. `type` 和 `file` 命令有什么区别?

2. 为什么每个目录中均有两个特殊的文件`."`和`.."`。

3. 绝对路径与相对路径有什么区别?

实验 2. 用户与权限

一、实验目的:

通过实验使学生进一步巩固并掌握用户管理和文件权限的设计方法。

二、实验要求:

1. 掌握用户组和用户的创建方法
2. 掌握文件权限的分配
3. 掌握进程访问文件的权限控制
4. 掌握扩展权限的使用

三、实验内容

1. 为系统添加新的登录用户

新建用户组和用户需超级用户权限，若为普通用户登录，需在 `groupadd`、`useradd` 和 `passwd` 命令前加上 `sudo` 前缀。

(1). 创建用户组 `student`、`music`、`football` 和 `project`。

```
$ sudo groupadd student           //添加 student 用户组
$ sudo groupadd music             //添加 music 用户组
$ sudo groupadd football          //添加 football 用户组
$ sudo groupadd project           // 创建项目组 jroject
```

(2). 创建用户 `zhangs` 和 `lis`。

```
$ sudo useradd zhangs -g student -G project ,music -md /home/zhangs
$ sudo useradd lis -g student -G project ,football -md /home/lis
```

为新建用户设置登录密码。

```
$ sudo passwd zhangs              // 为 zhangs 用户设置登录密码
$ sudo passwd lis                 //为 lis 用户设置登录密码
```

2. 建立共享目录

```
$ mkdir /usr/src/project          // 创建共享目录 project
$ chown :project /usr/src/project // 设置目录的所属用户组 student
# chmod g+rxw /usr/src/project    // 为目录的组用户设置所有权限
# chmod g+s /usr/src/project       // 为目录的组用户设置 SGID 标志位
```

3. 观察新用户创建目录和文件的权限

```
$ su zhangs                      // 切换至 zhangs 用户
$ umask 022                      // 设置新的权限掩码
$ cd /usr/src/project            // 进入共享目录
$ mkdir mydir                    // 新建目录 mydir
$ cat >myfile                    // 新建文件 myfile
$ ls -l                          // 观察新建文件权限
```

4. 扩展权限

```
$ su root                        // 切换至超级用户
$ chmod o+t /usr/src/project     // 为目录 project 设置 sticky 标志位
```

分别切换至 `zhangs` 和 `lis` 用户，在共享目录中分别创建一个文件，观察他们能否删除对方创建的文件？

5. 观察 `passwd` 命令的 SUID 标志位

```
$ ls -l /etc/shadow              // 观察密码文件的权限归属
```

```
-rw-r----- 1 root root 1090 Sep  7 19:10 shadow
$ ls -l /usr/bin/passwd // 观察 passwd 文件的 SUID 标志位
-rwsr-xr-x 1 root root 57972 May 17  2017 passwd
```

四、思考题

1. 简单阐述引入符号链接的目的。
2. 若某文件权限的字符表示为“rwsrw-r--”，写出其八进制表示。
3. 观察/usr/bin/sudo 文件的权限，分析权限分配的目的。
4. 一个项目团队由 prog1、proj2 和 proj3 三个成员构成，他们分别需要各自专属的目录，一个共享目录和一个临时目录，他们可在共享目录中操作其他人创建的文件，但在临时目录中仅能删除自己创建的文件，请在系统创建用户，并为他们创建相应的目录。

实验 3. 作业与进程

一、实验目的:

通过实验使学生进一步巩固并掌握作业控制和进程管理的相关概念，掌握常用工具的使用方法。

二、实验要求:

1. 掌握系统中用户进程的继承关系。
2. 掌握作业控制的方法。
3. 掌握会话、进程、进程组和控制终端的关系。
4. 掌握结束进程的方法。

三、实验内容

1. 观察系统中进程的父子关系

```
$ pstree // 显示用户进程的继承关系
```

2. 作业控制

打开第一个终端，在终端上输入下列命令。

```
$ (sleep 1000 & sleep 2000 )& // 将作业 1 置于后台
$ sleep 3000 | sleep 4000 // 将作业 2 置于前台
$ ctrl+z // 将前台作业 2 切换至后台
$ jobs // 观察作业的状态
$ bg %2 // 恢复作业 2 运行
$ fg %1 // 将作业 1 切换至前台
$ ctrl+z // 切换作业 1 至后台
```

打开第二个终端，在终端上输入下列命令。

```
S( sleep 5000 & sleep 6000) & // 将作业 1 置于后台
$ sleep 7000 | sleep 8000 & // 将作业 2 置于后台
```

3. 观察不同终端上作业中进程各类标识(SID,PID,PPID 和 PGID)的关系

```
$ ps -efj | grep sleep // 观察不同 ID 之间的关系
```

4. 结束进程

```
$ kill -TERM- xxx_id // xxx_id 表示某用户进程 ID
```

5. 重新启动系统

```
$ reboot // 重启系统
```

四、思考题

1. 控制终端为什么通常赋予前台进程组中的领头进程。
2. 普通用户可结束系统中任意正在运行的用户进程吗?

实验 4. 元字符与正则表达式

一、实验目的:

通过实验使学生进一步巩固并理解元字符和正则表达式的概念，掌握基本的使用方法。

二、实验要求:

1. 掌握元字符的概念及其使用。
2. 掌握编写基本的正则表达式。
3. 掌握文本过滤器 `egrep` 的使用方法。
- 4.

三、实验内容

1. Shell 环境下的通配符

新建文件 `a1,a2,a3,a*`,`ab` 和 `abc`，执行下列命令，观察执行结果。

```
$ ls a*           // 查找所有文件名以字符 a 开始的文件
$ ls a[12]        // 查找文件 a1 和 a2
$ ls a[1-3]       // 查找文件 a1,a2 和 a3
$ ls a\*          // 查找文件 a*
$ ls a?           // 查找文件名仅包含两个字符的文件，第一个字符为 a
```

2. 编写正则表达式，使用过滤器 `egrep` 命令，从文本文件中搜索所需的行。

```
$ egrep "^#" exam6-2.c // 过滤出所有以#开头的行
$ egrep ">$" exam6-2.c  // 过滤出所有以>结尾的行
$ egrep "\<m" exam6-2.c // 过滤出所有包含单词首字符为 m 的行
```

四、思考题

1. 列出当前目录下所有 C 源文件的文件名。
2. 使用 `egrep` 搜索包含 `ab,axyb,axyxyb,axyxyxyb...` 的行，写出相应的正则表达式。

实验 5 磁盘管理

一、实验目的:

通过实验使学生进一步巩固并理解磁盘管理的基本概念,掌握常用磁盘工具的使用方法。

二、实验要求:

1. 掌握块设备的分区方法。
2. 掌握在分区上建立文件系统。
3. 掌握文件系统的挂载/卸载。
4. 了解内 Linux 系统的引导加载过程。

三、实验内容

1. 管理一个新磁盘,在磁盘上建立分区并进行格式化。

(1). 在虚拟机上添加新的虚拟磁盘

- (a) 关闭虚拟机电源
- (b) 打开虚拟机设置,选择添加新硬件,进而添加新硬盘。
- (c) 重新启动虚拟机。

(2). 为新硬盘建立分区

假设新添加的虚拟硬盘为/dev/sdb

`$ fdisk /dev/sdb` / 为硬盘/dev/sdb 创建新的分区

(3). 在新分区上建立文件系统

`$ mkfs -t fat /dev/sdb1` // 在分区/dev/sdb1 上建立 fat 类型文件系统

(4). 使用新分区

`$ mount /dev/sdb1 /mnt` / 挂载分区/dev/sdb1 至/mnt 目录

`$ tar czvf code.tar.gz code` // 为目录 code 生成归档文件

`$ cp code.tar.gz /mnt` // 复制归档文件至新分区

`$ umount /mnt` / 卸载挂载点

2. 观察引导分区

`$ ls -l /boot` // 观察内核映像文件

`$ cat boot/grub` // 观察引导加载程序

`$ more grub.cfg` // 观察 grub 的引导配置文件

3. 观察 Linux 系统的启动过程

`$ reboot` // 重新启动计算机

`$ dmesg` / 观察系统启动信息

四、思考题

1. 如何在新磁盘上安装引导加载程序?
2. 如何为新分区设置 UUID 标识?

实验6 Shell 命令语言

一、实验目的:

通过实验使学生进一步巩固并掌握使用 Shell 命令语言编写脚本的方法。

二、实验要求:

1. 掌握 Shell 脚本的执行方法。
2. 掌握不同引号的作用。
3. 掌握不同变量的使用方法。
4. 掌握结构化命令的编程方法。

三、实验内容

1. echo 命令的使用

```
$ echo "a\tb\tc\nd\tf\n"           // 禁用转义符
$ echo -e "a\tb\tc\nd\tf\n"       // 启用转义符
```

2. 变量存储参数, 参数类型取决于操作。

```
$ var1=123 ; var2='123 '
$ test "$var1" -eq "$var2"         // 数学运算
$ echo $?                          // 测试结果
0                                  // 表示相等
$ test "$var1" = "$var2"           // 字符串运算
$ echo $?                          // 测试结果
1                                  // 表示不同
```

3. 观察 Shell 对不同引用的解释

```
$ var1="Hello Linux"
$ echo $var1                       // 显示$var1
$ echo "$var1"                     // 显示 Hello Linux
$ ls a*                             // 显示所有文件名以 a 开头的文件
$ ls a\*                           // 显示文件 a*
```

4. 命令置换

```
$ echo `pwd` 或 echo $(pwd)        // 显示命令 pwd 的执行结果
```

5. 命令的运行环境

(1) 在不同 Shell 环境下执行命令

```
$ var1=123 ; var2="hello Linux"    // 定义变量 var1 和 var2
$ export var2                      // 将变量 var2 转换为环境变量
$ (                                // 在子 Shell 环境执行复合命令
>echo $var1 $var2                  // 显示变量 var1 和 var2 的值
>var1=456                          // 修改变量 var1 的值为 456
>var2="How are you"               // 修改变量 var2 的值为 How are you
>)
123 Hello Linux                   // 显示复合命令的运行结果
echo $var1 $var2                  // 显示当前 Shell 环境中变量 var1 和 var2 的值
123 Hello Linux                   // 变量的值均未发生改变
$ {                                // 在当前 Shell 下执行复合命令
>echo $var1 $var2                 // 显示向 var1 和 var2 的值
```



```
> var1=456 // 修改变量 var1 的值为 456
>var2="How are you" // 修改变量 var2 的值为 How are you
>}
123 Hello Linux // 显示复合命令的运行结果
$ echo $var1 $var2 // 显示当前 Shell 环境中变量 var1 和 var2 的值
456 How are you // 变量 var1 和 var2 的值均发生了改变
```

(2) 在不同 Shell 环境下执行 Shell 脚本

编辑下列 Shell 脚本 demo.sh

```
#!/bin/bash
echo ${var1} ${var2}
var1=456 ; var2="How are you "

$ var1=123 // 设置变量 var1 的值为 123
$ export var2="Hello Linux" // 设置环境变量 var2 的值为 Hello Linux
$ bash demo.sh // 在子 Shell 环境下执行
    Hello Linux
$ echo ${var1} ${var2} // 显示当前 Shell 环境中变量 var1 和 var2 的值
123 Hello Linux
$ . demo.sh // 在当前环境下运行
123 Hello Linux
$ echo ${var1} ${var2} //显示当前 Shell 环境中变量 var1 和 var2 的值
456 How are you
```

6. Shell 预定义变量@和*的区别

编辑下列脚本 test.sh

```
#!/bin/bash
echo '---- @ ----'
for i in "$@"
do
    echo ${i}
done
echo '---- * ----'
for i in "$*"
do
    echo ${i}
done
```

```
$ bash test.sh 1 2 3 4 5 6 // 观察有什么不同
```

7. 运行书本上的实例脚本

(1) 运行书本上的实例 exam4-4.sh, 显示当前计算机事件

```
#!/bin/bash
hour=$(date +%H)
case ${hour} in
(0[5-9]|1[01])
    echo "Good morining"
;;
(1[2-7])
```

```
    echo "Good afternoon"
;;
(*)
    echo "Good evening "
;;
esac
```

(2). 运行书本上的实例 exam4-7.sh, 对输入数据进行排序

```
#!/bin/bash
# input data
if (( $#<1 ))
then
    echo "Usage: ${0} num1 num2..."
    exit 1
fi
i=0
for k in ${*}
do
    data[$i]=${k}
    ((i++))
done
# sort data
len=${#data[@]}
for (( i=0 ; i<$len-1 ; i++ ))
do
    for (( j=i+1 ; j<$len ; j++ ))
    do
        if [ ${data[$i]} -gt ${data[$j]} ]
        then
            tmp=${data[$i]}
            data[$i]=${data[$j]}
            data[$j]=${tmp}
        fi
    done
done
# display data
for k in ${data[*]}
do
    echo -n "${k} "
done
echo
```

四、思考题

1. 当前 Shell 环境和子 Shell 环境有什么不同?
2. 如何配置命令的搜索路径?

实验 7 GNU C 开发环境

一、实验目的:

通过实验使学生进一步巩固并理解 GNU C 软件开发的基本概念,掌握 GNU C 环境下各种工具的使用方法。

二、实验要求:

1. 掌握 gcc 工具的使用方法。
2. 掌握项目管理工具 make 目标依赖规则的编写方法。
3. 掌握静态库和共享库的创建和使用。

三、实验内容

1. gcc 命令的使用

编辑下列源代码文件 test.c, 以不同方式编译与链接。

```
// test.c
#include<stdio.h>
int count =20;
int main(void)
{
    int sum = 0;
#ifdef DEBUG
    printf("running in debug mode\n");
#else
    printf("running in no debug mode\n");
#endif
    for (int i = 0; i < count; i++)
        sum += i;
    printf("the sum is:\t%d\n", sum);
}
```

```
$ gcc -E -DDEBUG test.c -o test.i          // 预处理生成 test.i
$ gcc -S test.i                             // 编译生成 test.s
$ gcc -c test.s                             // 汇编生成 test.o
$ gcc test.o -o test                        // 链接生成 test
$ test                                     // 观察程序运行结果
running in debug mode
the sum is:    190
```

2. 软件项目管理, 编写 Makefile 脚本

假设一个小型软件项目, 经过系统分析与设计, 系统划分为三个模块, 它们对应的源代码文件分别为 lib1.c, lib2.c 和 semo.c, 其中 lib1.c 和 lib2.c 为基础函数库, 接口函数文件为 lib.h, demo.c 为主程序。

源文件 lib1.c

```
// lib1.c
int add(int x, int y){
```

```
    return x + y;
}
```

源文件 lib2.c

```
// lib2.c
int func(int count)
{
    int sum = 0;
    for ( int i = 0; i <= count; i++)
        sum += i;
    return sum;
}
```

源文件 lib.h

```
// lib.h
#ifndef _DEMOLIB_API_H
#define _DEMOLIB_API_H
    extern int add(int x, int y);
    extern int func(int count);
#endif
```

源文件 demo.c

```
// demo.c
#include<stdio.h>
#include "lib.h"
int main(void)
{
    int val;
    int x, y;
    x = 12;
    y = 18;
    val = add(x, y);
    printf("the mult of x and y is %d\n", val);
    val = func(100);
    printf("the sum is:\t%d \n", val);
    return 0;
}
```

(1) 准备工作

为上述源代码文件创建 project 目录，并将 lib1.c, lib2.c, lib.h 和 demo.c 置于该目录、

```
$ mkdir [project] // 创建项目工程目录
```

(2) 编写项目管理脚本 Makefile

(a) 编写规则时，详细给出具体规则。

脚本 Makefile

```
demo:demo.o lib1.o lib2.o
    gcc demo.o lib1.o lib2.o -o demo
demo.o:demo.c
    gcc -Wall -c demo.c
```

```
lib1.o:lib1.c
    gcc -Wall -c lib1.c
lib2.o:lib2.c
    gcc -Wall -c lib2.c
clean:
    rm -rf *.o demo
```

(b) 编写脚本时，使用变量、隐含规则和模式规则。

脚本 makefile

```
objs = demo.o lib1.o lib2.o
CFLAGS = -Wall
demo:$(objs)
    $(CC) $^ -o $@
clean:
    rm -rf $(objs)
```

(3) 生成可执行文件 demo

```
$ make -f Makefile           // 制定规则脚本 Makefile
$ ./demo                     // 执行 demo
$ make clean                  // 清理中间代码
$ make -f makefile           // 制定规则脚本 makefile
$ ./demo                     // 执行 demo
```

3. 静态库和共享库

(1) 静态库

(a) 创建静态库

```
$ gcc -Wall -c lib1.c        // 编译生成 lib1.o
$ gcc -Wall -c lib2.c        // 编译生成 lib2.o
$ ar -cUr libdemo.a lib1.o lib2.o // 归档生成静态库 libdemo.a
```

(b) 使用静态库

```
$ gcc -static -Wall demo libdemo.a -o demo // 使用静态库链接生成可执行程序 demo
$ gcc -static -Wall -L ./ demo.c -o demo -ldemo
```

(2) 共享库

(a) 创建共享库

```
$ gcc -fPIC lib1.c           // 编译生成地址无关目标文件 lib1.o
$ gcc -fPIC -c lib2.c        / 编译生成地址无关目标文件 lib2.o
$ gcc -shared lib1.o lib2.o -o libdemo.so // 链接生成共享库 libdemo.so
```

(b) 使用共享库

```
$ gcc -Wall demo.c libdemo.so -o demo // 使用共享库链接生成可执行程序 demo
$ gcc -Wall -L ./ demo.c -o demo -ldemo
```

四、思考题

1. 与使用 Shell 脚本相比，使用 make 工具管理软件项目有什么的优点？
2. 在嵌入式系统中，在链接生成可执行文件时，为什么通常使用静态库？

实验 8 信号处理

一、实验目的:

在理解信号工作机制的基础上，通过实验使学生进一步巩固并掌握信号编程的基本方法。

二、实验要求:

1. 掌握信号异步和同步的处理方法。
2. 掌握信号安全函数的概念和使用。

三、实验内容

1. 编写一程序，实现对信号 `SIGTERM` 的处理。做好系统关机前的善后工作。
2. 编写一程序，以同步方式处理到达的标准信号，假设到达的信号为 `SIGINT` 和 `SIGUSR1`。

四、思考题

1. 与标准信号相比，实时信号有哪些特点？
2. 件数内核引入信号文件的作用。

实验 9 进程控制

一、实验目的:

在理解子进程实现机制的基础上，通过实验使学生进一步巩固并掌握进程控制的编程方法。

二、实验要求:

1. 掌握创建子进程的编程方法。
2. 掌握加载可执行程序的编程方法。
3. 掌握父子进程间的同步控制。
4. 了解守护进程的实现技术。

三、实验内容

1. 实现一个简单的 Shell 程序，循环执行用户输入的外部命令，。要求基于 `fork()`和 `execve()` 核心函数构建。不支持变量和控制命令等复杂的语法成分。
2. 编写一个守护进程，管理用户的登录时间，对超市用户发出警告，必要时结束会话。

四、思考题

1. Linux 内核中，采用 COW 算法实现内存复制的优点？
- 2/ 内核为什么要引入 `wait()/waitpid()`接口函数？

实验 10 多线程技术

一、实验目的:

在理解线程概念的基础上, 通过实验使学生进一步巩固并掌握线程编程的基本方法。

二、实验要求:

1. 掌握线程的创建方法、
2. 掌握多线程同步控制技术。
3. 在多线程环境下, 学会使用线程安全函数。

三、实验内容

1. 编写一程序, 实现基于多线程的文件复制。
2. 利用线程同步技术, 实现 m 个生产者和 n 个消费者间的同步。

演示代码如下所示。

```
#include <pthread.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <ctype.h>
struct thread_info {
    pthread_t thread_id;
    int      thread_num;
};
struct thread_info *tinfo;
struct thread_info *cust;
pthread_mutex_t lock ;
pthread_cond_t notempty , notfull ;
int  nprocd = 0, nmax = 10;
void *producer(void* arg)
{
    struct thread_info *tinfo = arg;
    while (1) {
        pthread_mutex_lock(&lock);
        while (nprocd == nmax)
            pthread_cond_wait(&notfull, &lock);
        sleep(rand() % 4);
        nprocd ++;
        printf("-----   %d \t count %d   \n", tinfo->thread_num,nprocd);
        pthread_mutex_unlock(&lock);
        pthread_cond_signal(&notempty);
        sleep(rand() % 4);
    }
}
```



```

        pthread_exit((void *)0);
    }
    void *consumer(void * arg)
    {
        struct thread_info *tinfo = arg;
        while (1) {
            pthread_mutex_lock(&lock);
            while (nproc == 0)
                pthread_cond_wait(&notempty, &lock);
            sleep(rand() % 4);
            nproc--;
            printf("=====\t count %d\t \n", tinfo->thread_num, nproc);
            pthread_mutex_unlock(&lock);
            pthread_cond_signal(&notfull);
            sleep(rand() % 4);
        }
        pthread_exit((void *)0);
    }

    int main(int argc, char *argv[])
    {
        int opt;
        int stack_size = 2048*1024;
        int nthread = 2;
        int mcust = 2;
        while ((opt = getopt(argc, argv, "s:p:c:")) != -1) {
            switch (opt) {
                case 's':
                    stack_size = atol(optarg)*1024;
                    break;
                case 'p':
                    nthread = atol(optarg);
                    break;
                case 'c':
                    mcust = atol(optarg);
                    break;
                default:
                    fprintf(stderr, "Usage:%s [-s stack-size(kb)] [-n num of threads] \n", argv[0]);
                    exit(1);
            }
        }
        pthread_attr_t attr;
        pthread_attr_init(&attr);
        pthread_attr_setstacksize(&attr, stack_size);
        pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
    }

```

```
pthread_attr_setschedpolicy(&attr, SCHED_FIFO);
struct sched_param param;
param.sched_priority = 2;
pthread_attr_setschedparam(&attr, &param);
pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);
pthread_mutex_init(&lock, NULL);
pthread_cond_init(&notempty, NULL);
pthread_cond_init(&notfull, NULL);
tinfo = calloc(nthread, sizeof(struct thread_info));
cust = calloc(mcust, sizeof(struct thread_info));
for (int i = 0; i < nthread; i++) {
    tinfo[i].thread_num = i + 1;
    pthread_create(&tinfo[i].thread_id, &attr, &producer, &tinfo[i]);
}
for (int i = 0; i < mcust; i++) {
    cust[i].thread_num = i + 1;
    pthread_create(&cust[i].thread_id, &attr, &consumer, &cust[i]);
}
while(1);
pthread_attr_destroy(&attr);
free(tinfo);
pthread_exit(NULL);
}
```

四、思考题

1. 与进程相比，线程有哪些优缺点？
2. 如何判断一个函数为线程安全函数？

实验 1.1 网络编程

一、实验目的:

在理解网络协议的基础上，通过实验使学生进一步巩固并掌握网络编程的基本方法。

二、实验要求:

1. 掌握基于 TCP 客户机/服务器的编程方法。
2. 掌握基于 UDP 客户机/服务器的编程方法。
3. 掌握使用多进程/多线程技术，构建并发服务器。

三、实验内容

1. 使用循环迭代模式实现一个基于 UDP 的时间服务器，考虑支持不同时区。
2. 实现一个基于多进程的客户机/服务器，支持文件的上传和下载。

四、思考题

1. 与普通关闭连接相比，优雅关闭有什么优点？
2. 当调用 send 函数时，若成功返回，是否意味着数据已全被对方接收？

实验 1.12 异步 I/O

一、实验目的:

在理解异步 I/O 概念的基础上,通过实验使学生进一步巩固并掌握各种异步 I/O 的编程方法。

二、实验要求:

1. 掌握信号驱动的异步 I/O 编程方法。
2. 掌握基于 `select()/poll()` 函数的多路复用编程。
3. 掌握基于 `epoll` 的异步 I/O 编程方法。
- 4.

三、实验内容

1. 实现一个基于异步 I/O 信号驱动的时间服务器。
2. 使用 `epoll` 技术,实现一个基于文字的聊天服务器。

四、思考题

1. 与多进程/多线程的并发处理相比,简述异步 I/O 的特点。
2. 与传统 `select/poll` 相比, `epoll` 有什么优势?